ASSIGNMENT- 03

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Database Design Analysis

By
Name: MERIN THOMAS
Reg. No.: 2511021240062

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

1. INTRODUCTION

The Paranormal Investigation Agency Management System is designed to help an investigation agency manage its cases efficiently. It stores information about clients, investigators, locations, case categories, equipment, evidence, reports, and payments.

The database replaces scattered notebooks and case files with a structured system. It helps the agency assign investigators, track equipment, maintain evidence records, prepare reports, and manage payments.

This assignment analyzes the database design by identifying data redundancy, insertion anomalies, update anomalies, deletion anomalies, and functional dependencies. It also explains normalization and suggests improvements to make the database more reliable and easier to maintain.

Objectives

- Identify possible data redundancy in the database.
- Explain insertion, update, and deletion anomalies with examples.
- Identify functional dependencies between attributes.
- Analyze the database using normalization principles.
- Recommend improvements to the existing database design.

2. OVERVIEW OF THE DATABASE DESIGN

The proposed database contains 11 tables.

Table Name	Purpose
Clients	Stores client contact details.
Investigators	Stores information about agency investigators.
Locations	Stores information about investigation locations.
Case_Categories	Stores categories of paranormal activity.
Equipment	Maintains the agency's equipment inventory.
Cases	Stores the main details of each investigation.
Case_Investigator	Connects cases with assigned investigators.
Case_Equipment	Records equipment used for investigations.

Evidence	Stores evidence collected during investigations.
Reports	Stores investigation findings and final reports.
Payments	Stores payment details for investigation services.

The Cases table acts as the central table and connects clients, locations, and case categories. The Case_Investigator and Case_Equipment tables manage relationships between cases, investigators, and equipment.

The remaining tables maintain specific information about evidence, reports, and payments.

3. DATA REDUNDANCY

Data redundancy occurs when the same information is unnecessarily stored in multiple places. It increases storage requirements and may cause inconsistencies when information is updated.

3.1 Examples of Data Redundancy

1. Repeated client details

If a client reports multiple cases, storing the client's name, email, and phone number in every case record would repeat the same information.

Improvement: Store client information once in the Clients table and use client_id to connect the client with different cases.

2. Repeated investigator details

An investigator may work on several cases. Storing the investigator's name, phone number, and specialization in every case record would create unnecessary duplication.

Improvement: Store investigator details in the Investigators table and use the Case_Investigator table to record assignments.

3. Repeated category details

Several cases may belong to the same category. Repeating the category name and description in every case record creates redundancy.

Improvement: Store category information in Case_Categories and reference it through category_id in the Cases table.

4. Repeated equipment details

The same equipment item may be used in multiple investigations. Repeating its name, condition, and purchase date for every case would duplicate information.

Improvement: Maintain equipment information in the Equipment table and record its use in Case_Equipment.

The proposed database design reduces these forms of redundancy by storing different types of information in separate tables.

4. INSERTION, UPDATE, AND DELETION ANOMALIES

Database anomalies are problems that may occur when unrelated information is stored together or when the same information is repeated unnecessarily.

4.1 Insertion Anomaly

An insertion anomaly occurs when information cannot be added without inserting unrelated information.

Example:

Suppose the agency purchases a new EMF meter but has not assigned it to any investigation. If equipment details are stored only in a combined table that requires a case ID, the agency may be unable to register the equipment independently.

Effect: Equipment information becomes dependent on the creation of an investigation record.

Solution: Store the new equipment directly in the Equipment table. Create a Case_Equipment record when the equipment is assigned to a case.

4.2 Update Anomaly

An update anomaly occurs when the same information appears in multiple rows and must be updated everywhere.

Example:

Suppose an investigator's phone number is stored in five different case records. When the phone number changes, all five records must be updated. If one record is missed, the database will contain inconsistent information.

Effect: The database may store different phone numbers for the same investigator.

Solution: Store the investigator's phone number only in the Investigators table and connect the investigator to cases using investigator_id.

4.3 Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes other important information.

Example:

Suppose investigator details and case assignments are stored in the same table. If the only case assigned to an investigator is deleted, the investigator's contact information may also be lost.

Effect: Deleting a case may accidentally remove information that should be retained.

Solution: Store investigator details separately in the Investigators table. Deleting a case should not automatically delete the investigator's master record.

4.4 Summary of Anomalies

Anomaly	Cause	Example	Solution
Insertion anomaly	Unrelated information is stored together.	New equipment cannot be added without a case.	Maintain a separate Equipment table.
Update anomaly	The same information is repeated.	An investigator's phone number must be changed in several rows.	Store contact details once in Investigators.
Deletion anomaly	Deleting one record removes another required fact.	Deleting a case removes the only stored copy of investigator details.	Separate investigator details from case records.

5. FUNCTIONAL DEPENDENCIES

A functional dependency describes a relationship between attributes in a table. It means that one attribute or a group of attributes uniquely determines another attribute.

It is represented as:

X → Y

This means that the value of X determines the value of Y.

The following dependencies assume that each identifier is a primary key and uniquely identifies its corresponding record.

5.1 Functional Dependencies in the Main Tables

Table	**Functional Dependency**
Clients	client_id → name, email, phone, address, registration_date
Investigators	investigation_id → name, specialization, phone, email, experience_level, date_joined
Locations	location_id → address, location_type, year_built, known_history
Case_Categories	category_id → category_name, description
Equipment	equipment_id → equipment_name, equipment_type, condition_status, purchase_date
Cases	case_id → client_id, location_id, category_id, date_reported, investigation_date, case_status, urgency_level
Evidence	evidence_id → case_id, recorded_by, evidence_type, description, file_reference, timestamp
Reports	report_id → case_id, prepared_by, final_verdict, summary, report_date
Payments	payment_id → case_id, amount, payment_mode, payment_status, payment_date

Note: The original database design uses investigation_id as the primary key of the Investigators table. This assignment retains that name to remain consistent with Assignment 1. The same name should be used consistently in later SQL assignments.

5.2 Functional Dependencies in Junction Tables

Case_Investigator

The composite primary key consists of case_id and investigation_id.

(case_id, investigation_id) → role_in_case

This means that the combination of a case and an investigator determines the investigator's role in that case, assuming each investigator has only one role per case.

Case_Equipment

If an equipment item is checked out at most once for a particular case, the functional dependency is:

(case_id, equipment_id) → checkout_time, return_time

However, if the same equipment can be checked out multiple times for the same case, this combination may not uniquely identify each checkout. In that situation, a separate checkout_id should be introduced as the primary key.

6. NORMALIZATION ANALYSIS

Normalization is the process of organizing data into appropriate tables to reduce unnecessary duplication and prevent insertion, update, and deletion anomalies.

The main normal forms considered in this assignment are First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

6.1 Example of a Poorly Designed Table

Consider the following example of a combined table that stores case and client information together.

case_id	client_name	client_phone	category_name	location_address
C101	Anita Roy	9876501001	EVP Session	12 Lake Road
C102	Anita Roy	9876501001	Residential Haunting	12 Lake Road
C103	Rohan Das	9876501002	EVP Session	44 Hill Street

In this table, Anita Roy's phone number and address are repeated because she has multiple cases.

If her phone number changes, multiple rows must be updated. If a client has not reported a case, storing the client's details may also become difficult. Deleting the last case belonging to a client could remove the only stored copy of that client's details.

These problems show why separating client, case, location, and category information is important.

6.2 First Normal Form (1NF)

A table is in First Normal Form when:

- Each cell contains a single, atomic value.
- There are no repeating groups or lists within a field.
- Each record can be uniquely identified.

Example:

Storing several investigator IDs in a single cell of the Cases table would violate 1NF.

Improvement:

Store each case-investigator assignment as a separate record in the Case_Investigator table. Each row should contain one case ID and one investigator ID.

This ensures that individual assignments can be stored and managed separately.

6.3 Second Normal Form (2NF)

A table is in Second Normal Form when:

- It is already in 1NF.
- Every non-key attribute depends on the entire candidate key, not just part of a composite key.

Example:

In the Case_Investigator table, the composite key is:

(case_id, investigation_id)

The role_in_case attribute depends on the combination of the case and investigator because an investigator may have different roles in different cases.

However, an investigator's name and phone number depend only on investigation_id, not on the complete composite key.

Improvement:

Keep role_in_case in Case_Investigator and store investigator details in the Investigators table. This avoids partial dependencies and unnecessary duplication.

6.4 Third Normal Form (3NF)

A table is in Third Normal Form when:

- It is already in 2NF.
- Non-key attributes do not depend transitively on a candidate key through another non-key attribute.

Example:

Suppose the Cases table stores case_id, category_id, and category_name.

The category name depends on category_id rather than directly on case_id. If several cases use the same category, the category name will be repeated.

Improvement:

Store category_id and category_name in the Case_Categories table. The Cases table should store category_id as a foreign key.

Similarly, client contact information should be stored in Clients, while Cases should reference the client through client_id.

This reduces repeated information and helps maintain data consistency.

7. RECOMMENDED NORMALIZED STRUCTURE

The proposed database separates different entities into appropriate tables.

Information	**Table**	**Relationship**
Client name, email, phone, and address	Clients	Cases references Clients using client_id.
Investigator details	Investigators	Case_Investigator references Investigators using investigation_id.
Category name and description	Case_Categories	Cases references Case_Categories using

		category_id.
Location address and history	Locations	Cases references Locations using location_id.
Equipment details and condition	Equipment	Case_Equipment references Equipment using equipment_id.
Case dates, status, and urgency	Cases	Related records reference Cases using case_id.
Evidence details	Evidence	Evidence references Cases using case_id.
Final investigation findings	Reports	Reports references Cases using case_id.
Payment details	Payments	Payments references Cases using case_id.

This structure reduces unnecessary duplication by keeping each type of information in its appropriate table.

The design should also be checked against the actual business rules before implementation to ensure that all keys and relationships are correct.

8. PROPOSED IMPROVEMENTS

8.1 Standardize Identifiers and Attribute Names

Use consistent names for primary keys and foreign keys throughout the database. For example, the identifier investigation_id should be used consistently for investigators because that is the name defined in Assignment 1.

Correct spelling mistakes in attribute names, such as speciliazation to specialization and File_refernce to file_reference.

Consistent naming makes SQL queries easier to understand and reduces implementation errors.

8.2 Define Primary Keys and Foreign Keys

Every table should have a primary key that uniquely identifies each record.

Foreign keys should connect related tables and prevent references to records that do not exist.

For example:

- Cases.client_id should reference Clients.client_id.
- Cases.location_id should reference Locations.location_id.
- Cases.category_id should reference Case_Categories.category_id.
- Evidence.case_id should reference Cases.case_id.
- Payments.case_id should reference Cases.case_id.

These constraints help maintain referential integrity.

8.3 Apply Integrity Constraints

Appropriate constraints should be applied to ensure data validity.

- NOT NULL: Prevents required fields from being left empty.
- UNIQUE: Prevents duplicate values where uniqueness is required.
- CHECK: Restricts values to valid conditions, such as non-negative payment amounts.
- DEFAULT: Supplies a predefined value when an optional value is not provided.
- FOREIGN KEY: Ensures that a referenced record exists in the related table.

These constraints should reflect the actual requirements of the agency.

8.4 Clarify Report and Payment Relationships

The original design specifies that each case has exactly one final report and one payment record.

If these rules are retained, the database should enforce them appropriately. For example, Reports.case_id and Payments.case_id can be made unique to prevent multiple records for the same case.

If the agency later needs report versions or multiple payments, such as advance and final payments, the relationships should be revised to allow multiple records per case.

The design must match the agency's actual requirements.

8.5 Improve Equipment Checkout Tracking

The Case_Equipment table should accurately record equipment checkout and return times.

If the same equipment can be checked out more than once for a case, a checkout_id primary key should be added to identify each checkout event separately.

A constraint should also ensure that the return time is not earlier than the checkout time.

8.6 Validate Dates and Amounts

The database should prevent invalid data wherever possible.

For example:

- Payment amounts should not be negative.
- Return times should not be earlier than checkout times.
- Case statuses should use approved values.
- Payment statuses should use valid values such as Pending, Paid, or Refunded.
- Required dates and identifiers should not be left empty.

Validation improves the accuracy of the stored data.

8.7 Protect Evidence References

The Evidence table should maintain a clear file_reference for each evidence file.

Access to evidence records should be controlled because investigation records may contain private client information. The agency should also ensure that referenced files are stored securely and remain available when needed.

9. SUMMARY OF RECOMMENDATIONS

Issue	**Recommended Improvement**	**Expected Benefit**
Repeated client and investigator information	Store each entity's details in its own table.	Reduces redundancy.
Inconsistent attribute names	Standardize identifiers and spelling.	Makes SQL easier to understand.
Unclear junction-table keys	Use suitable composite keys or a separate checkout_id.	Identifies records correctly.
Unclear report and payment rules	Define and enforce the required relationships.	Keeps the design consistent with business requirements.
Invalid values and dates	Apply appropriate integrity constraints.	Improves data accuracy.
Poorly managed evidence references	Maintain secure and valid file references.	Improves evidence management.

10. CONCLUSION

The Paranormal Investigation Agency Management System uses 11 tables to manage clients, investigators, locations, categories, equipment, cases, assignments, evidence, reports, and payments.

Separating these entities into appropriate tables helps reduce data redundancy and prevents insertion, update, and deletion anomalies. Functional dependencies help identify how attributes depend on primary keys, while normalization principles help organize the database efficiently.

The database can be improved further by using consistent attribute names, defining appropriate primary and foreign keys, applying integrity constraints, and clarifying the business rules for reports, payments, and equipment checkout.

These improvements will make the database more consistent, reliable, and easier to maintain.